

State Space Models

Set up

```
library(fpp3)
library(tidyverse)
library/slider)
library(gridExtra)
```

1. Finding code

Look at the CRAN task views page to find what packages are available for state space models.

2. Local linear trend model

A local linear trend model is defined by the equations

$$X_t = X_{t-1} + 0.2 + W_t,$$

$$Y_t = X_t + V_t.$$

We can interpret this as the time series (Y_t) comprising a “trend” component, modeled as a random walk with drift, and a random component (V_t).

- a. How can one write this in the form of a state space model?
- b. Simulate a time series of length 50 from this model and plot the results.
- c. Use the functions `SSModel()` and `SSMcustom()` in the `KFAS` package to specify a state space model. Read the documentation on how to do this.
- d. Use `KFS()` to performing filtering, smoothing, and one-step-ahead forecasting for (X_t) . Read the documentation to understand the output of the function.

- e. Plot the three time series against the true (X_t) values and the observed values (Y_t). Also plot level 0.95 confidence bands.
- f. What are the differences between the three time series?
-

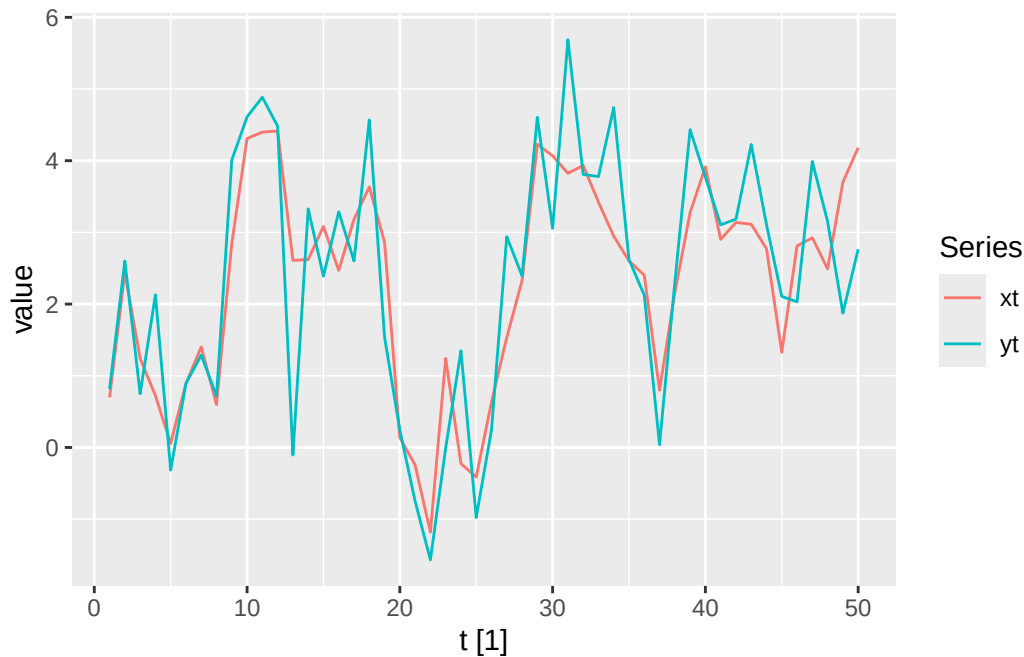
Can write

$$\begin{bmatrix} X_t \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X_{t-1} \\ 0.2 \end{bmatrix} + \begin{bmatrix} W_t \\ 0 \end{bmatrix}$$

$$Y_t = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} X_t \\ 0.2 \end{bmatrix} + V_t$$

```
set.seed(5209)
n <- 50
lt_dat <- tibble(t = 1:n,
                 wt = rnorm(n),
                 vt = rnorm(n),
                 xt = cumsum(wt) + 0.2*t,
                 yt = xt + vt)

lt_dat |>
  as_tsibble(index = t) |>
  pivot_longer(cols = c(xt, yt), names_to = c("Series")) |>
  autoplot(value)
```



We first encode the parameters of the model.

```
alpha <- 0.2
Phi <- matrix(c(1, 0, 1, 1), nrow = 2)
A <- matrix(c(1, 0), nrow = 1)
Q <- matrix(c(1, 0, 0, 0), nrow = 2)
R <- 1
```

We use `SSMCustom()` and `SSModel()` from the `KFAS` package to specify the state space model. To find out how to do this, we can consult the package documentation. Note the difference in notation.

```
library(KFAS)
```

Please cite `KFAS` in publications by using:

Jouni Helske (2017). `KFAS`: Exponential Family State Space Models in R. *Journal of Statistical Software*.

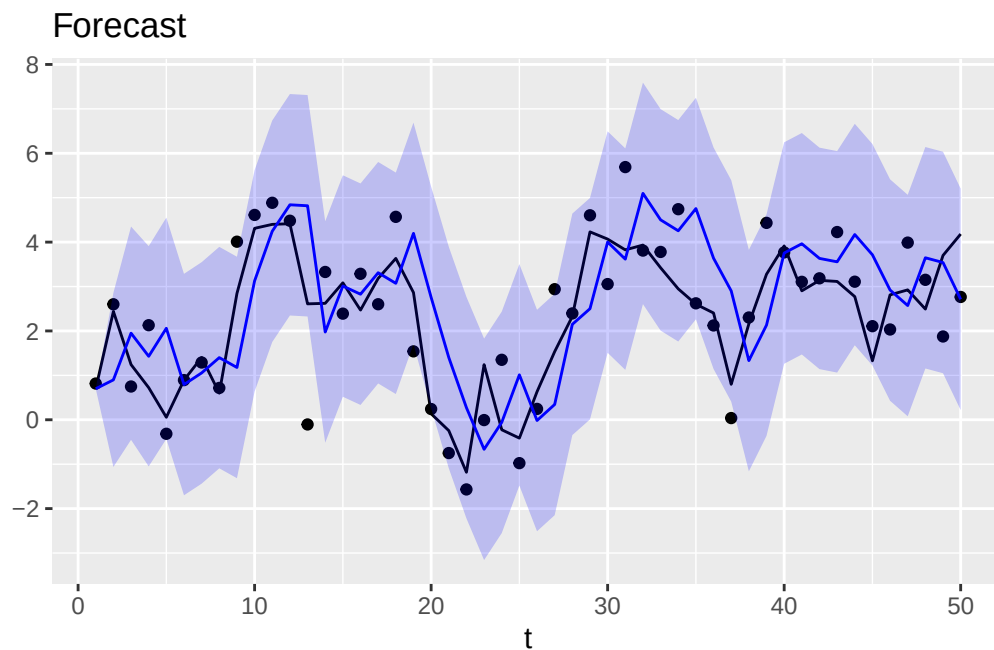
```
lt_model <- SSModel(as.ts(lt_dat$yt) ~ SSMcustom(
  T = Phi, Z = A, Q = Q, R = diag(c(1, 1)),
  a1 = matrix(c(lt_dat$xt[1], alpha), nrow = 2)) - 1,
  H = R
)
```

```
lt_out <- KFS(lt_model)
```

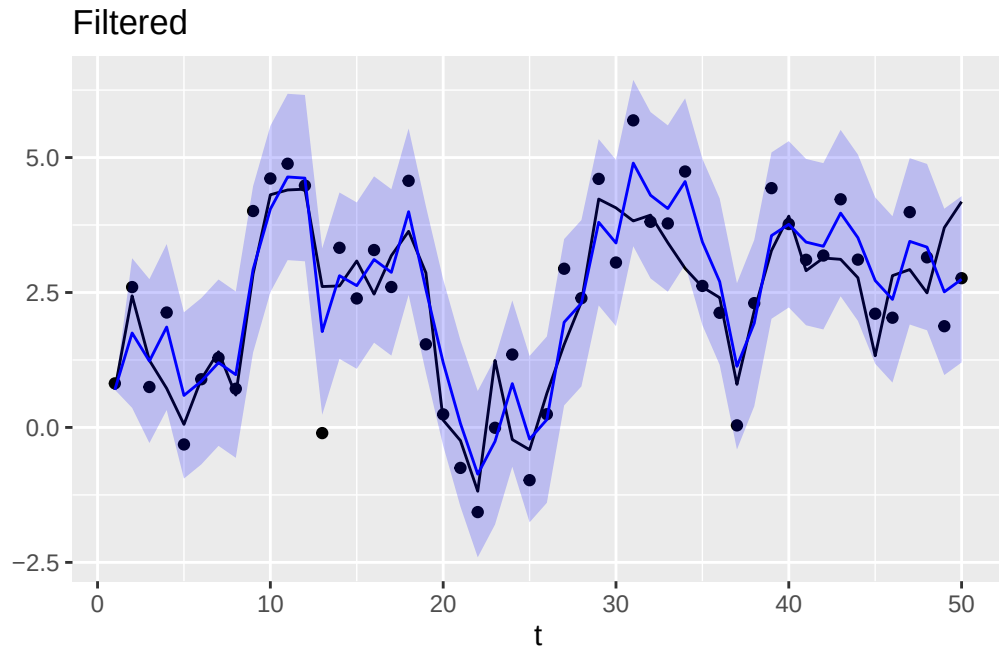
```
# Construct a tibble with all the required data
```

```
lt_dat_new <-
  tibble(lt_dat,
    forecast = lt_out$a[-(n+1), 1],
    filtered = lt_out$att[, 1],
    smoothed = lt_out$alphahat[, 1],
    forecast_var = lt_out$P[1, 1, ][-(n+1)],
    filtered_var = lt_out$Ptt[1, 1, ],
    smoothed_var = lt_out$V[1, 1, ]
  )
```

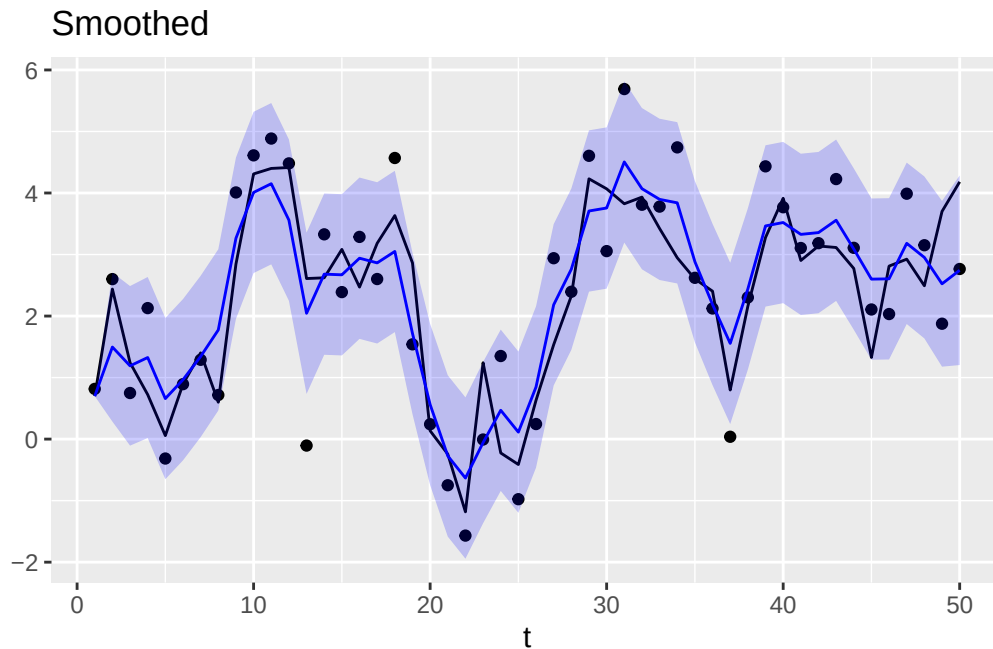
```
lt_dat_new |>
  ggplot(aes(x = t)) +
  geom_point(aes(y = yt)) +
  geom_line(aes(y = xt)) +
  geom_line(aes(x = t, y = forecast), color = "blue") +
  geom_ribbon(aes(ymin = forecast - 1.96 * sqrt(forecast_var),
    ymax = forecast + 1.96 * sqrt(forecast_var)), fill = "blue", alpha = 0.2) +
  ylab("") + ggtitle("Forecast")
```



```
lt_dat_new |>
  ggplot(aes(x = t)) +
    geom_point(aes(y = yt)) +
    geom_line(aes(y = xt)) +
    geom_line(aes(x = t, y = filtered), color = "blue") +
    geom_ribbon(aes(ymin = filtered - 1.96 * sqrt(filtered_var),
                  ymax = filtered + 1.96 * sqrt(filtered_var)), fill = "blue", alpha = 0.2) +
    ylab("") + ggtitle("Filtered")
```



```
lt_dat_new |>
  ggplot(aes(x = t)) +
    geom_point(aes(y = yt)) +
    geom_line(aes(y = xt)) +
    geom_line(aes(x = t, y = smoothed), color = "blue") +
    geom_ribbon(aes(ymin = smoothed - 1.96 * sqrt(smoothed_var),
                  ymax = smoothed + 1.96 * sqrt(smoothed_var)), fill = "blue", alpha = 0.2) +
    ylab("") + ggtitle("Smoothed")
```



Across all three plots, we see that the forecast, filtered, and smoothed values are closer than the observations Y_t to the true values X_t . Furthermore, filtered values make use of more information and are thus more accurate than forecasts. Similarly, smoothed values make use of the most information, and are marginally more accurate than filtered values.

3. Global temperature

Consider the two time series `gtemp_land` and `gtemp_ocean` discussed in the lecture video. Load, and normalize these time series via the following code snippet.

```
library(astsa)
temp_dat <- tibble(ocean = gtemp_ocean / sd(gtemp_ocean),
                  land = gtemp_land / sd(gtemp_land))
```

- What is an appropriate state space model for these time series?
- What are the parameters to be estimated?
- Fit the parameters of the model using `fitSSM()`.
- Use the fitted model to perform smoothing, and plot the resulting values, together with a level 0.95 confidence band and the two original time series.

An appropriate choice of model is

$$\begin{bmatrix} X_t \\ \alpha \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X_{t-1} \\ \alpha \end{bmatrix} + \begin{bmatrix} W_t \\ 0 \end{bmatrix}$$
$$\begin{bmatrix} Y_{t1} \\ Y_{t2} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} X_t + \begin{bmatrix} V_{t1} \\ V_{t2} \end{bmatrix}.$$

Parameters to be estimated are the the initial state, the state noise covariance Q , and the observation noise covariance R . To indicate that these are estimated, we fill them in using NA.

```
init_state <- matrix(c(NA, NA), nrow = 2)
Phi <- matrix(c(1, 0, 1, 1), nrow = 2)
Q <- matrix(c(NA, 0, 0, 0), nrow = 2)
A <- matrix(c(1, 1, 0, 0), nrow = 2)
R <- diag(c(NA, NA))

temp <- temp_dat |> as.matrix()
gtemp_model <- SSMModel(temp ~ SSMcustom(T = Phi, Z = A, Q = Q, R = diag(c(1, 1))) - 1, H = R)

gtemp_update <- function(pars, model) {
  model["Q"] <- matrix(c(exp(pars[1]), 0, 0, 0), nrow = 2)
  model["H"] <- diag(c(exp(pars[2]), exp(pars[3])))
  model["a1"] <- matrix(c(exp(pars[4]), exp(pars[5])), nrow = 2)
  model
}

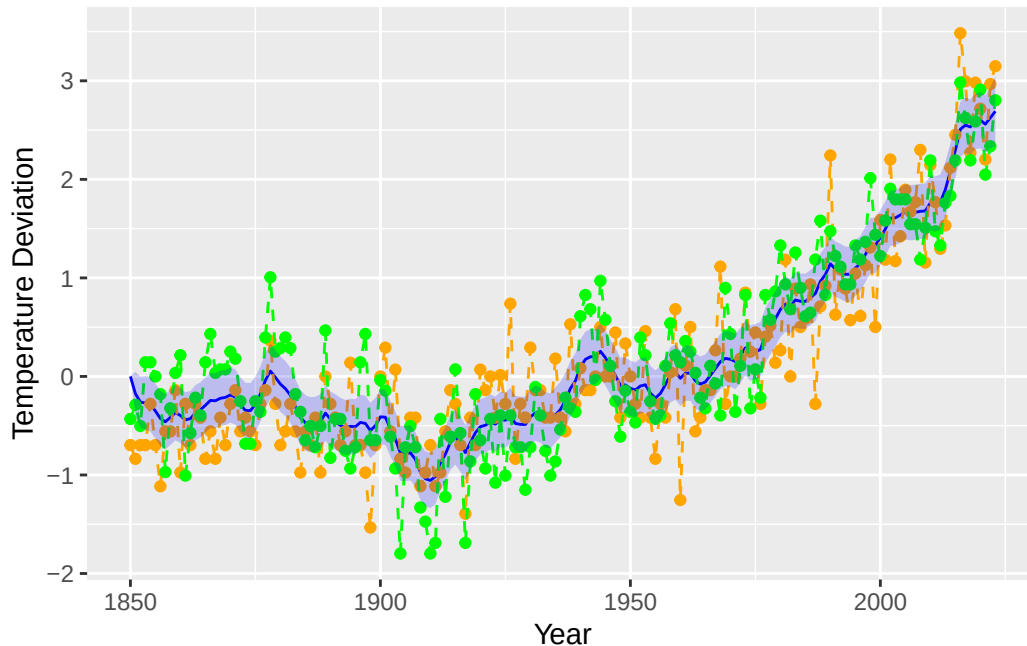
gtemp_fit <- fitSSM(gtemp_model, updatefn = gtemp_update, inits = rep(0, 5), method = "BFGS")

gtemp_out <- KFS(gtemp_fit$model)

tibble(Year = 1850:2023, temp_dat,
       smoothed = gtemp_out$alphahat[, 1],
       smoothed_var = gtemp_out$V[1, 1, ]) |>
  ggplot(aes(x = Year)) +
  geom_line(aes(y = smoothed), color = "blue") +
  geom_point(aes(y = land), color = "orange") +
  geom_line(aes(y = land), color = "orange", linetype = "dashed") +
  geom_point(aes(y = ocean), color = "green") +
```

```
geom_line(aes(y = ocean), color = "green", linetype = "dashed") +
geom_ribbon(aes(ymin = smoothed - 1.96 * sqrt(smoothed_var),
              ymax = smoothed + 1.96 * sqrt(smoothed_var)),
          fill = "blue", alpha = 0.2) +
ylab("Temperature Deviation")
```

Don't know how to automatically pick scale for object of type <ts>. Defaulting to continuous.



4. Biomarker monitoring

Load the biomarker dataset using the following code snippet.

```
biomarker <-
  tibble(Day = 1:length(WBC),
         WBC = WBC,
         PLT = PLT,
         HCT = HCT) |>
  as_tsibble(index = Day) |>
  mutate(across(everything(), ~replace(., . == 0, NA)))
```


- What is an appropriate state space model for these time series?
- What are the parameters to be estimated?
- Fit the parameters of the model using `fitSSM()`.
- Use the fitted model to perform filtering, and plot the resulting values, together with a level 0.95 confidence band and the two original time series.
- Interpret the fitted parameter values for Φ .
- Why do our results look different compared to those in Figure 6.6 in Shumway and Stoffer (page 315)?

An appropriate model is

$$\begin{bmatrix} X_{t1} \\ X_{t2} \\ X_{t3} \end{bmatrix} = \begin{bmatrix} \phi_{11} & \phi_{12} & \phi_{13} \\ \phi_{21} & \phi_{22} & \phi_{23} \\ \phi_{31} & \phi_{32} & \phi_{33} \end{bmatrix} \begin{bmatrix} X_{t-1,1} \\ X_{t-1,2} \\ X_{t-1,3} \end{bmatrix} + \begin{bmatrix} W_{t1} \\ W_{t2} \\ W_{t3} \end{bmatrix}.$$

$$\begin{bmatrix} Y_{t1} \\ Y_{t2} \\ Y_{t3} \end{bmatrix} = A_t \begin{bmatrix} X_{t1} \\ X_{t2} \\ X_{t3} \end{bmatrix} + \begin{bmatrix} V_{t1} \\ V_{t2} \\ V_{t3} \end{bmatrix}.$$

```
biomarker_dat <- biomarker |> as_tibble() |> select(-Day) |>
  as.matrix()
T <- matrix(NA, 3, 3) # state transition
Z <- diag(1, 3) # observation matrix
R <- diag(1, 3) # noise embedding
Q <- diag(NA, 3) # state noise
H <- diag(0, 3) # obs noise
a1 <- biomarker_dat[1, ] # initial state

biomarker_model <- SSMModel(biomarker_dat ~
  SSMcustom(Z = Z, T = T, R = R,
    Q = Q, a1 = a1) - 1, H = H)
biomarker_update <- function(pars, model) {
  model["T"] <- matrix(pars[1:9], nrow = 3)
  model["Q"] <- diag(exp(pars[10:12]))
  model
}

T0 <- c(1, 0, 0, 0, 1, 0, 0, 0, 1)
```

```

Q0 <- rep(0, 3)
inits <- c(T0, Q0)

biomarker_fit <- fitSSM(biomarker_model,
                        updatefn = biomarker_update,
                        inits = inits, method = "BFGS")
biomarker_out <- KFS(biomarker_fit$model, filtering = "state",
                     smoothing = "state")

```

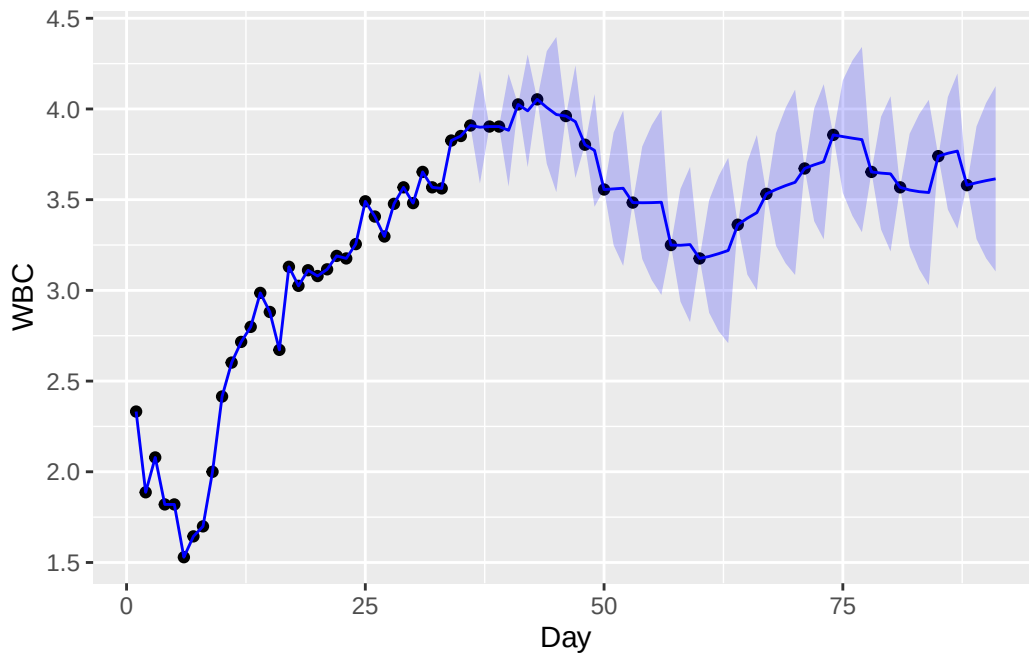
```

plt1_dat <-
  tibble(Day = 1:91,
         WBC = biomarker$WBC,
         filtered = biomarker_out$att[, 1],
         filtered_var = biomarker_out$Ptt[1, 1, ])

plt1_dat |>
  ggplot(aes(x = Day)) +
  geom_point(aes(y = WBC), color = "black") +
  geom_line(aes(y = filtered), color = "blue") +
  geom_ribbon(aes(ymin = filtered - 1.96 * sqrt(filtered_var),
                ymax = filtered + 1.96 * sqrt(filtered_var)),
            fill = "blue", alpha = 0.2)

```

Warning: Removed 37 rows containing missing values or values outside the scale range (`geom\_point()`).

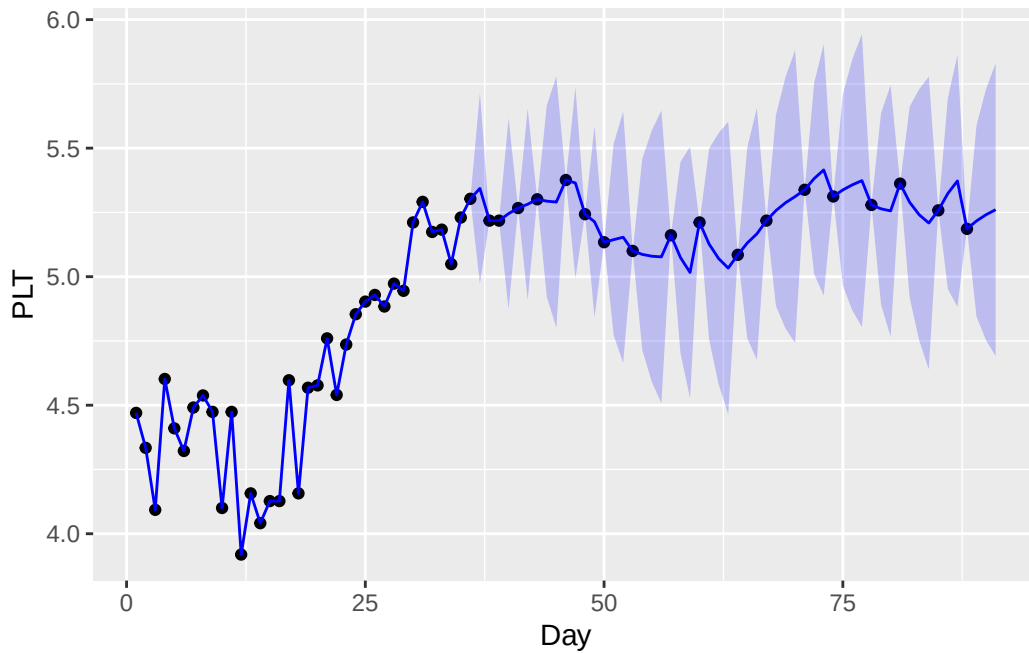


PLT

```
plt2_dat <-
  tibble(Day = 1:91,
    PLT = biomarker$PLT,
    filtered = biomarker_out$att[, 2],
    filtered_var = biomarker_out$Ptt[2, 2, ])

plt2_dat |>
  ggplot(aes(x = Day)) +
    geom_point(aes(y = PLT), color = "black") +
    geom_line(aes(y = filtered), color = "blue") +
    geom_ribbon(aes(ymin = filtered - 1.96 * sqrt(filtered_var),
      ymax = filtered + 1.96 * sqrt(filtered_var)),
      fill = "blue", alpha = 0.2)
```

Warning: Removed 37 rows containing missing values or values outside the scale range (``geom_point()``).

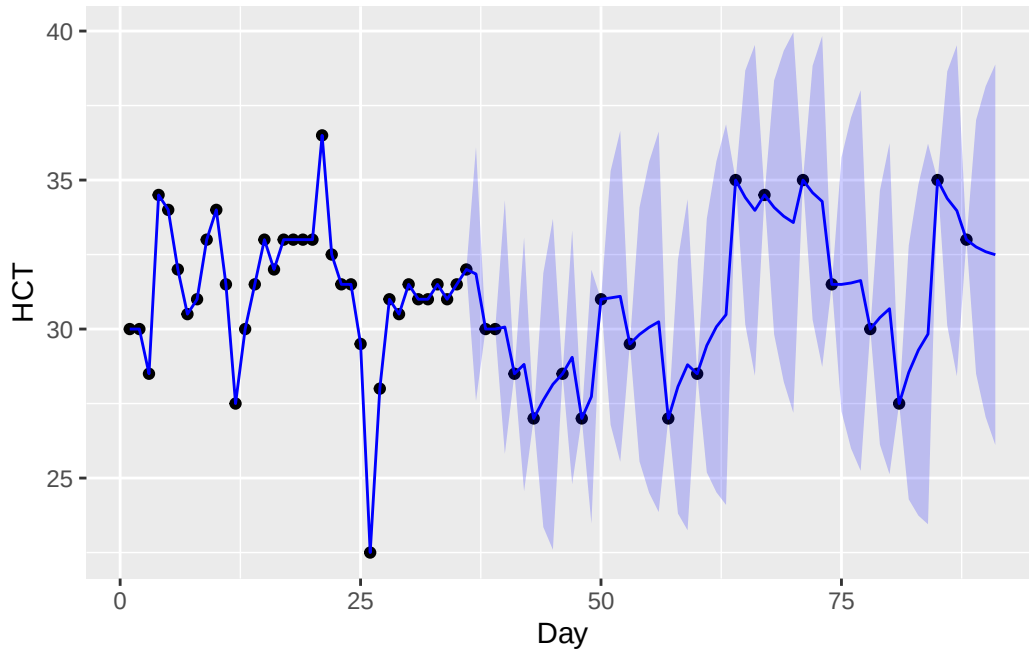


HCT

```
plt3_dat <-
  tibble(Day = 1:91,
    HCT = biomarker$HCT,
    filtered = biomarker_out$att[, 3],
    filtered_var = biomarker_out$Ptt[3, 3, ])

plt3_dat |>
  ggplot(aes(x = Day)) +
    geom_point(aes(y = HCT), color = "black") +
    geom_line(aes(y = filtered), color = "blue") +
    geom_ribbon(aes(ymin = filtered - 1.96 * sqrt(filtered_var),
      ymax = filtered + 1.96 * sqrt(filtered_var)),
      fill = "blue", alpha = 0.2)
```

Warning: Removed 37 rows containing missing values or values outside the scale range (`geom\_point()`).



We may view the fitted transition matrix Φ as follows:

```
biomarker_out$model$T
```

```
, , 1
```

	custom1	custom2	custom3
custom1	0.9449866	0.005792947	0.00546266
custom2	0.1277343	0.833640410	0.01322103
custom3	-0.8587830	1.682623084	0.82133278

To interpret this, recall that the bottom row implies that

$$X_{t,3} = -0.859X_{t-1,1} + 1.673X_{t-1,2} + 0.821X_{t-1,3} + W_t,$$

so that the HCT value depends positively on the previous values of PLT and HCT, but negatively on the previous value of WBC.